Distributed Sensor Hub

Final Report for the Distributed Systems Course 2025/2026

Alex Santini
alex.santini2@studio.unibo.it

April 24, 2026

Distributed Sensor Hub is a fully decentralized peer-to-peer platform for smart-city sensor data collection and replication, designed to operate across multiple nodes without any central coordinator. Each node may host multiple local sensors, ingest their readings through a pluggable sensor interface to support different sensor types, and maintain a local replica of the shared system state.

The platform disseminates sensor updates through push-pull gossip and resolves conflicts with deterministic timestamp-based Last-Writer-Wins merging, providing eventual consistency across replicas. In the demo setup, sensor streams are simulated for reproducibility, but the same ingestion model can be extended to real devices through dedicated adapters.

To remain operational in realistic distributed environments, the system combines gossip-based membership, phi-accrual failure detection, and anti-entropy recovery after crashes or temporary network partitions. In CAP terms, the project prioritizes availability and partition tolerance over strong consistency, while still guaranteeing convergence once communication is restored.

Fault-injection experiments on multi-node deployments show stable recovery and consistent replica alignment after failures and partitions, supporting the suitability of the approach for decentralized smart-city monitoring scenarios.

Disclaimer

During the development of the project and the preparation of this report, the author used Large Language Models (LLMs) as auxiliary tools for limited coding support, brainstorming, and improving the clarity and structure of the written text.

All outputs produced with the assistance of LLMs were critically reviewed, validated, and, when necessary, revised by the author. All architectural decisions, system design choices, and implementation details were defined and verified by the author, who takes full responsibility for the final content of this report.

1 Concept

Product type. The developed software is a distributed experimental platform composed of a set of autonomous peer-to-peer services and an optional web application for monitoring. Concretely, it combines:

- a set of autonomous peer-to-peer nodes responsible for sensor ingestion, local state management, membership handling, failure detection, and replicated state synchronization;
- an optional read-only web/introspection layer for monitoring and runtime observability.

Use case description. The software manages a cluster of sensor nodes that ingest local readings, replicate sensor state across peers, and expose runtime information for monitoring. The project is intentionally designed as a controlled distributed-systems laboratory rather than as a production-ready smart-city platform. This makes it possible to focus on decentralization, eventual consistency, membership evolution, failure detection, anti-entropy recovery, and observability under faults and network partitions.

A typical execution proceeds as follows. First, a configurable number of nodes are started. Each node may host one or more local sensors and begins ingesting readings through a pluggable sensor interface. In the current demonstration setup, sensor streams are simulated for reproducibility, but the same interface can be extended to real devices through dedicated adapters. As readings are produced, nodes disseminate updates through push-pull gossip, exchange heartbeat evidence, propagate membership information, and synchronize their replicated state. Conflicts are resolved deterministically through a *Last-Write-Wins* policy based on $(ts\_ms, origin)$, so replicas progressively converge after message delays, crashes, or temporary partitions.

Typical users are smart-city monitoring stakeholders with two main human interaction profiles:

- *observational interaction* (control-room operators): inspect live sensor values, node health, and cluster evolution from monitoring workstations;

- *operational interaction* (technical operators/developers): deploy nodes, tune configuration parameters, and execute recovery or fault-handling procedures from development or administration machines.

Users are typically located wherever the system is operated or observed, such as development environments, control-room settings, or laboratory test setups. Interaction is usually periodic rather than continuous: operators interact with the system when deploying experiments, changing configuration, injecting faults, or checking recovery, while observers access the dashboard whenever live status inspection is needed. In addition, external tools may consume the exposed API to inspect, collect, or further process the data maintained by the system.

Interaction channels and devices:

- HTTP API for programmatic interaction, monitoring, and integration with external tools;
- desktop/laptop web browser for optional dashboard-based monitoring.

Data handling and roles:

- the system does not store personal user data; it stores replicated sensor-state records, timestamps, origin identifiers, membership/liveness metadata, topology views, and introspection metrics/events as node-local runtime state;
- optional log files provide operational traces for debugging and validation on the machines running the nodes;
- main roles include: *node process* (autonomous peer), *cluster operator/developer*, and *observer client* (dashboard, tests, or external tooling).

Why distribution is needed. Distribution is a core requirement, not an implementation detail:

- autonomous data collection at multiple nodes rather than at a central coordinator;
- temporary divergence of local replicas before eventual convergence;
- decentralized membership management and peer liveness assessment through heartbeat observation and phi-accrual failure detection;
- resilience under crashes, restarts, and temporary network partitions through gossip dissemination and anti-entropy recovery;
- realistic smart-city deployment assumptions, where sensing points are naturally distributed at the edge.

The system does not focus on long-term historical analytics or centralized query processing. It focuses on the information needed to study decentralized sensor-state replication: current sensor values, timestamps, origin identifiers, membership and liveness metadata, topology knowledge, and runtime observability data.

2 Requirements Elicitation and Analysis

Requirements were elicited from the intended project goal described in section 1: build a decentralized laboratory platform for distributed sensor ingestion, replicated state propagation, fault observation, and monitoring. The elicitation process therefore focused on the observable services the platform must provide to operators and observers, on the quality attributes expected from a distributed-system prototype, and on the practical constraints imposed by course evaluation and reproducible experimentation.

2.1 Glossary

- **Node**: autonomous peer process participating in the cluster and executing the full runtime stack.
- **Peer**: another node known by a node through bootstrap, heartbeat exchange, or gossip dissemination.
- **Sensor reading**: atomic observation produced by a local sensor source, including sensor identifier, value, timestamp, and origin.
- **Replicated sensor state**: node-local view of the cluster-wide latest known sensor values after merge.
- **Membership view**: node-local knowledge about known peers and their liveness status.
- **Observer**: human user or external tool consuming read-only monitoring data.
- **Operator**: human user responsible for starting nodes, configuring experiments, and injecting or observing failures.
- **Convergence**: condition in which reachable replicas expose the same winning sensor values after enough communication.
- **Partition**: temporary communication disruption that prevents some nodes from exchanging messages with the rest of the cluster.

2.2 Functional Requirements

FR1 – Multi-node autonomous operation. The system shall allow multiple nodes to start independently and participate in the same distributed deployment without requiring a central coordinator.

Acceptance criterion. When a cluster of at least two configured nodes is started, each node becomes operational as an autonomous process and exposes its local service interfaces without depending on a single master node.

FR2 – Local sensor ingestion. Each node shall be able to host one or more local sensor sources and ingest their readings into the distributed system state.

Acceptance criterion. Given at least one configured sensor on a node, the node eventually exposes fresh sensor records associated with its own origin through the read API or introspection API.

FR3 – Cluster-wide sensor-state dissemination. The system shall propagate sensor updates among nodes so that readings produced on one node become visible on other reachable nodes.

Acceptance criterion. If a node generates new sensor readings while communication paths to other nodes are available, those readings eventually appear in the replicated sensor-state view exposed by the other reachable nodes.

FR4 – Deterministic conflict resolution. The system shall resolve concurrent or conflicting updates in a deterministic way so that replicas can converge to the same winning value for each sensor record.

Acceptance criterion. When two or more updates compete for the same logical sensor record, all nodes that receive the same set of updates eventually expose the same winner for that record.

FR5 – Membership discovery and peer knowledge propagation. The system shall allow nodes to discover peers and maintain a distributed view of cluster membership.

Acceptance criterion. After startup and sufficient message exchange, a node exposes a membership view containing the peers it has discovered directly or indirectly.

FR6 – Liveness observation. The system shall provide operators with a node-local assessment of peer liveness so that missing or degraded peers can be observed during experiments.

Acceptance criterion. If a known peer becomes unreachable for long enough, at least one observing node eventually reports a degraded or failed status for that peer in its membership/introspection output.

FR7 – Recovery after crashes or temporary disconnections. The system shall allow a node that restarts or reconnects after temporary unavailability to recover the replicated state needed to rejoin normal operation.

Acceptance criterion. After a node crash or temporary isolation, once communication is restored the recovering node eventually exposes a replicated state aligned with the rest of the reachable cluster.

FR8 – Read-only observability interface. The system shall provide a read-only programmatic interface for observing sensor state, membership, topology-related information, events, and operational metrics.

Acceptance criterion. A client can query documented HTTP endpoints and obtain structured snapshots of the node's current distributed-system view without modifying runtime state.

FR9 – Human-oriented monitoring support. The system shall support human observation of the running cluster through a dashboard or equivalent visual monitoring interface.

Acceptance criterion. An observer can inspect live cluster information from a web browser by connecting to the exposed read-only API of a node.

2.3 Non-Functional Requirements

NFR1 – Decentralization. Normal operation shall not depend on any central coordinator or single control component.

Acceptance criterion. The system continues to ingest local readings, maintain local state, and serve read-only observability data on surviving nodes even if one arbitrary peer becomes unavailable.

NFR2 – Eventual consistency. The replicated sensor state shall provide eventual, not immediate, consistency across nodes.

Acceptance criterion. In the absence of new conflicting updates and after communication is restored, reachable replicas converge to the same winning values for the same sensor records.

NFR3 – Partition tolerance with degraded semantics. The system shall remain available for local operation during temporary network partitions, accepting that different partitions may temporarily diverge.

Acceptance criterion. During a partition, nodes in each connected component continue local ingestion and observation; after the partition heals, their replicas eventually converge again.

NFR4 – Fault observability. The system shall make failures and recovery visible enough to support distributed-systems experimentation and debugging.

Acceptance criterion. Operators can inspect status changes, replication activity, and recovery evidence through exposed membership, event, or metric views.

NFR5 – Reproducibility of experiments. The platform shall support repeatable validation scenarios on development machines and course evaluation environments.

Acceptance criterion. Using the documented setup, an operator can start a predefined topology and reproduce representative convergence and recovery experiments without relying on external managed services.

NFR6 – Extensibility of sensor sources. The system shall be open to additional sensor types without changing the conceptual role of the rest of the distributed runtime.

Acceptance criterion. New sensor sources can be introduced as additional producers while preserving the same downstream ingestion, replication, and observation capabilities.

NFR7 – Operational scalability at course-project scale. The platform shall support small and medium experimental clusters rather than only a fixed two-node demo.

Acceptance criterion. The documented deployment can be instantiated with multiple nodes and still provides replicated state and observability functions for the whole experiment.

2.4 Implementation Constraints

IR1 – Self-contained deployment. The project shall be executable in a self-contained way on commodity machines used for development and evaluation, without requiring paid or institution-managed cloud infrastructure.

Justification. This constraint follows from course-project reproducibility and portability requirements.

Acceptance criterion. The system can be started locally with the provided repository artifacts and documented dependencies.

IR2 – Reproducible multi-node setup. The project shall provide a reproducible way to instantiate representative multi-node deployments for demonstrations and validation.

Justification. The report and validation activities require repeatable distributed scenarios, including fault-injection experiments.

Acceptance criterion. An operator can launch a predefined topology from the repository and repeat the same experiment configuration with consistent system roles and interfaces.

2.5 Relevant Distributed System Features

Not all classical distributed-system features have the same priority in this project. Since *Distributed Sensor Hub* is a decentralized monitoring and experimentation platform, some features are central design drivers, while others are intentionally secondary.

Transparency. Only partially relevant. The system hides low-level transport details from observers, but it does *not* aim to hide the fact that it is distributed. On the contrary, replication lag, topology evolution, liveness suspicion, and recovery dynamics are part of what the project wants to make observable. Failure transparency is therefore limited by design.

Fault tolerance and dependability. Highly relevant. Nodes may crash, restart, or become temporarily disconnected, and the platform is expected to continue operating on surviving nodes while later recovering convergence. Availability and recoverability are more important than uninterrupted global consistency. Integrity is also relevant in the sense that replicas must converge deterministically rather than drift indefinitely.

Scalability. Relevant, but at moderate scale. The project is not meant for Internet-scale deployments; however, it must scale beyond a trivial two-node example and remain meaningful on multi-node laboratory topologies. The main concern is preserving understandable behavior as the number of peers and sensor updates grows.

Security and trust. Only marginally relevant in the current scope. The platform does not manage personal data or adversarial multi-tenant access, and the main users are trusted operators and observers in controlled environments. Security hardening is therefore not a primary project driver, although it remains a possible future extension.

Resource sharing. Relevant. Nodes share distributed knowledge rather than centralized hardware resources: replicated sensor winners, membership evidence, topology

views, and operational metadata. Coordination is required to keep those shared views convergent despite asynchrony and failures.

Openness, interoperability, and heterogeneity. Relevant. The project benefits from clean interfaces between sensor producers, node runtime, and observation clients. This matters because simulated sensors are only one current source of data, while the same ingestion boundary can later support heterogeneous adapters and external tools.

Evolvability and maintainability. Highly relevant. As a course project intended to demonstrate multiple distributed-systems concerns, the platform must remain understandable, modular, and extensible, especially for adding new sensors, observability surfaces, and validation scenarios.

Performance, concurrency, and communication efficiency. Relevant, but not under hard real-time constraints. The system executes many concurrent activities such as sensor production, replication, heartbeats, gossip exchange, and HTTP observation. Efficiency matters because excessive communication can obscure convergence behavior, but the project does not target strict latency guarantees.

Economy and costs. Relevant mainly as a practical constraint. The project should run on ordinary development hardware and avoid external infrastructure dependencies, which supports reproducibility, portability, and low evaluation cost.

3 Design

This chapter describes the high-level design adopted to satisfy the requirements in section 2. The central design goal is to support autonomous multi-node execution (FR1), cluster-wide sensor-state dissemination (FR3), deterministic convergence (FR4), recovery after failures (FR7), and strong observability (FR8–FR9), while preserving decentralization (NFR1), eventual consistency (NFR2), and partition tolerance (NFR3).

3.1 Architecture

The system follows a **fully decentralized peer-to-peer architecture** in which every node executes the same logical stack and can both produce local sensor data and consume data generated by other peers. There is no master node, no dedicated coordinator, and no centralized database. This architectural style directly addresses FR1 and NFR1: any node can participate in the cluster as an autonomous peer, and the failure of one peer does not invalidate the whole system.

From a logical viewpoint, the architecture is organized into three cooperating planes:

- a **control plane**, responsible for peer discovery, membership maintenance, liveness observation, and topology knowledge;
- a **data plane**, responsible for local sensor ingestion, local state maintenance, and state dissemination among peers;
- an **observability plane**, responsible for exposing cluster snapshots, metrics, and monitoring views to operators and observer clients.

This separation is important because the project intentionally distinguishes between *knowing who is in the system* and *replicating what the system knows about sensors*. Membership and liveness information evolve under different timing assumptions and serve different purposes than sensor-state replication. Keeping the two concerns separate improves modularity, clarity, and evolvability, which supports NFR6 and the maintainability goals discussed in section 2.5.

Within each node, the architecture is further layered:

- a **transport layer**, dealing only with network communication;
- a **protocol layer**, defining message types and dispatching received messages to the correct logic;
- a set of **domain components**, where the actual semantics of membership, failure detection, replication, topology, and observation are applied.

This layered organization keeps low-level communication concerns separate from distributed-system semantics. As a consequence, the same conceptual design would still hold if the underlying transport or encoding technology changed, which is consistent with the intended role of the design chapter.

3.2 Infrastructure

The infrastructure is intentionally minimal. The essential infrastructural unit is the **node process**, replicated one time for each participant in the cluster. Every node contains the same internal subsystems: sensor ingestion, local state management, peer membership handling, failure detection, peer-to-peer protocol handling, and a read-only HTTP observation interface. This homogeneous-node model simplifies deployment and supports both FR1 and NFR5, because the same runtime can be instantiated in different topologies without role specialization.

No external infrastructural component is required for normal operation. In particular, the design does not rely on:

- a central database;
- a message broker;
- a coordination service;
- a load balancer;
- a dedicated monitoring backend.

The only optional external-facing component is the browser dashboard, which is not part of the distributed core and behaves as a read-only observer. It can be hosted separately because it only consumes the observation API exposed by any node.

The distributed deployment model is therefore the following:

- one or more nodes run on different processes, containers, or machines;
- each node may host a different local set of sensors;
- nodes communicate directly with peers over the inter-node network;
- observers may connect to any node's read-only web interface or HTTP API.

Component discovery is bootstrapped through a configured set of initial peers. After first contact, discovery is no longer purely static: nodes can exchange peer knowledge and extend their local membership view. Naming is based on stable node identifiers plus network coordinates required for communication. This choice is sufficient for the project scope because it avoids introducing centralized service-discovery infrastructure while still allowing evolving cluster knowledge, which is coherent with FR5 and NFR1.

From a physical standpoint, the design does not assume geographic distribution at continental scale; however, it does assume that nodes may be distributed across separate machines or containers and may suffer message delay, temporary unreachability, or partial connectivity. This is exactly the failure model needed to study NFR2 and NFR3.

3.3 Modelling

The main domain entities are:

- **Node**, representing an autonomous distributed participant;
- **Peer**, representing another known node together with its membership and liveness metadata;
- **Sensor source**, representing a local producer of readings;
- **Sensor reading**, representing one observation generated by a sensor source;
- **Replicated sensor record**, representing the current winning value known for one logical sensor identifier;
- **Membership entry**, representing a node's local belief about another peer's status;
- **Topology entry**, representing adjacency knowledge useful for network observability and connection policies;
- **Observer view**, representing an aggregated, read-only projection of internal distributed state.

These entities map naturally to infrastructural components. Sensor sources are local to a single node, while replicated sensor records, membership entries, and topology entries are stored as node-local replicas. In other words, the design does not centralize domain state in one authoritative repository; instead, each node stores its own best-effort replica of the distributed knowledge relevant to operation and observation.

The most important domain events are:

- production of a local sensor reading;
- discovery of a new peer;
- reception of a heartbeat;
- transition of a peer between liveness states;
- emission or reception of a sensor-state update;
- request for missing replication data;
- full-state transfer during recovery;
- generation of an introspection snapshot for observers.

The exchanged messages can be grouped into three categories:

- **control messages**, for discovery, membership dissemination, and liveness observation;

- **state-replication messages**, for sensor update propagation and catch-up;
- **read-only queries**, for HTTP observation by users or tools.

The system state comprehends:

- latest winning sensor values together with origin and timestamp metadata;
- membership knowledge about known peers;
- liveness evidence and peer-status information;
- topology knowledge and connection-related views;
- operational events and metrics used for observability.

This model is intentionally centered on *current replicated knowledge*, not on long-term historical storage. That design keeps the project focused on convergence, liveness, and fault handling rather than on archival analytics.

3.4 Interaction

There are two main interaction families in the system.

The first is **peer-to-peer interaction among nodes**. This interaction is asynchronous and message-based. It is used for:

- bootstrap and peer discovery;
- heartbeat exchange;
- dissemination of membership knowledge;
- dissemination and retrieval of replicated sensor state;
- recovery synchronization after missed updates.

The second is **observer interaction**, where external clients query a node through the HTTP read API. This interaction is request-response and read-only. It does not participate in cluster coordination and it never mutates the distributed state.

At the inter-node level, communication follows a layered path:

> *domain intent* $\rightarrow$ *protocol message* $\rightarrow$ *transport delivery* $\rightarrow$ *protocol dispatch* $\rightarrow$ *domain update*

The main interaction patterns are:

- **bootstrap request/response**, used when a node joins and needs initial peer knowledge;
- **periodic heartbeat exchange**, used to collect liveness evidence;

- **gossip dissemination**, used to spread membership knowledge without central coordination;
- **push-pull state synchronization**, used to exchange recent sensor-state updates efficiently;
- **full synchronization on demand**, used when incremental recovery is insufficient;
- **polling-based observation**, used by the dashboard and external tools to inspect cluster state.

This design supports FR3, FR5, FR6, FR7, and FR8 because it combines low-coupling peer communication with an observation interface that stays orthogonal to the replicated core.

3.5 Behaviour

The behavior of the main components can be summarized as follows.

Sensor sources are active components: they periodically generate local readings and inject them into the node. They are conceptually producers and do not need global knowledge.

The local state manager is stateful and authoritative for the node's replica. It receives local and remote updates, applies deterministic merge rules, stores the current winners, and exposes snapshots to both replication and observability components. This component is central to FR2, FR3, and FR4.

The membership manager is also stateful. It stores known peers, updates direct or indirect liveness evidence, and maintains the node's local view of cluster membership.

The failure detector behaves as a local evaluator rather than as a replicated authority. It interprets heartbeat timing and produces suspicion levels for peers observed directly by the node. Those local judgements are then translated into membership information that can be disseminated.

The heartbeat and gossip runtime is periodic. At each round, it evaluates liveness, sends heartbeat probes, and disseminates updated membership knowledge. Its purpose is not to guarantee exact membership synchronization at every instant, but to make peer-status knowledge progressively converge.

The replication runtime is periodic as well. It pushes recent sensor-state updates to selected peers and pulls missing updates from others. When incremental recovery is not sufficient, it can trigger a broader synchronization phase. This behavior is essential for FR7 and NFR2.

The observability subsystem is read-only. It collects snapshots from state, membership, topology, and event/metric providers, then publishes a coherent observation view for users and tools.

State updates are therefore concentrated in a small set of stateful components:

- local sensor-state storage;

- local membership storage;
- local topology and observability stores.

Other components mainly trigger transitions or move data between these stores. This separation keeps write paths explicit and supports maintainability.

3.6 Data and Consistency Issues

The system stores several forms of node-local runtime data:

- the replicated sensor-state winners;
- the incremental replication view needed for propagation and recovery;
- the membership table and liveness metadata;
- topology knowledge;
- recent events and metrics for introspection.

This data is primarily **runtime state**, not long-term persistent storage. The design does not require an external database because the goal is to study distributed replication and convergence of current system knowledge rather than durable business records. This keeps the architecture lightweight and self-contained, matching IR1.

Conceptually, the replicated sensor state behaves like a key-value map from a logical sensor identifier to the current winning record. The important property is not relational querying power, but deterministic merge semantics. For this reason, the core consistency problem is solved at the application level rather than delegated to a storage engine.

Consistency is handled differently for different data types:

- **sensor state** uses eventual consistency with deterministic Last-Writer-Wins conflict resolution based on timestamp and origin metadata;
- **membership state** also converges eventually, but it reflects local observations that may temporarily differ across nodes;
- **observability data** is inherently approximate and time-sensitive, because it represents a snapshot of changing distributed state.

This means the system explicitly rejects strong consistency as a primary goal. During partitions or transient delays, different nodes may expose different but still valid local views. Once communication resumes and no newer conflicting updates appear, the replicas converge. This choice is aligned with NFR2 and NFR3.

Data is shared among components in well-defined ways:

- sensor producers share generated readings with the local state manager;
- the state manager shares snapshots and deltas with the replication runtime and the observability subsystem;

- the membership subsystem shares peer-status information with heartbeat, gossip, topology, and observability components;
- the observability subsystem reads from other stores but does not modify them.

This read/write asymmetry is deliberate: observation must remain decoupled from mutation so that FR8 can be satisfied without introducing accidental feedback paths.

3.7 Fault-Tolerance

Fault tolerance is one of the central design drivers of the project. The first mechanism is **state replication**: sensor-state knowledge is disseminated among peers so that the system does not depend on a unique owner of cluster knowledge. Replication is best-effort and decentralized, which supports NFR1 and NFR3 even though it does not provide instantaneous consistency.

The second mechanism is **heartbeat-based liveness observation**. Nodes periodically exchange liveness probes and evaluate the timing of received heartbeats. This allows each node to form a local judgement about whether a peer is alive, suspected, or dead. Since such knowledge is local and timing-dependent, the design treats liveness as a continuously revised estimate rather than as an absolute truth.

The third mechanism is **membership gossip**. A node does not keep liveness knowledge only for itself; it disseminates updated membership information so that peer knowledge can propagate even when direct connectivity patterns are incomplete. This improves resilience of membership awareness and helps the system maintain a broader view of the cluster.

The fourth mechanism is **anti-entropy recovery**. Incremental propagation is efficient when peers stay mostly synchronized, but it may be insufficient after outages, restart, or missed traffic. For this reason, the design includes a stronger recovery path that allows a lagging node to regain a coherent replicated view after disruption, directly supporting FR7.

Error handling is intentionally failure-aware rather than failure-oblivious. If a component or peer becomes temporarily unavailable:

- surviving nodes continue local operation;
- local replicas may diverge temporarily;
- membership views reflect degraded reachability;
- synchronization resumes when communication becomes possible again.

This behavior is preferable to halting the whole system because the project prioritizes availability and observation under faults over strong global coordination.

3.8 Availability

The design does not introduce classical web-style caching layers or external load balancers, because the system is not centered on high-volume client traffic toward a centralized service. Instead, availability is achieved structurally through replication, autonomy of nodes, and local read access to each node's current view.

Each node acts as a locally available observation point. As long as a node remains running, it can still:

- ingest its own local sensor readings;
- maintain its local replica;
- expose read-only snapshots to operators and tools.

In this sense, replicated state itself plays a role analogous to availability-oriented caching: each node keeps a local working copy of shared cluster knowledge, so read operations do not require contacting a central authority.

No explicit load balancing is required because there is no single mandatory ingress point for observation. Different observers may query different nodes, and each node exposes its own local view. This is sufficient for the experimental scope of the project.

During a **network partition**, the design favors continued local operation over blocking. Nodes inside each connected component continue to exchange information among themselves, while nodes separated by the partition stop receiving fresh updates from one another. Consequently:

- sensor-state replicas may diverge temporarily across partitions;
- membership views may mark remote peers as suspected or dead;
- local observation remains available inside each connected component.

When the partition heals, incremental dissemination and recovery mechanisms progressively re-align the replicas. This behavior is a direct consequence of choosing availability and partition tolerance over strong consistency, and it is exactly the semantics required by NFR2 and NFR3.

3.9 Security

Security is intentionally limited in the current design because, as discussed in section 2.5, it is not a primary driver of this project. The system is assumed to run in controlled laboratory, development, or trusted demonstration environments.

Accordingly, the current design does **not** make authentication a core architectural dependency for either peer-to-peer traffic or read-only observation endpoints. Similarly, it does not define a rich authorization model with differentiated user permissions. The effective roles are simple: nodes participate in the distributed protocol, while observers query read-only information.

This choice keeps the architecture aligned with the educational scope of the project and avoids obscuring the main distributed-systems concerns with security infrastructure that is orthogonal to the report goals. However, the design has been kept modular enough that stronger security policies could be introduced later at clear boundaries:

- on node-to-node channels, to authenticate peers and protect inter-node communication;
- on HTTP observation endpoints, to restrict access to monitoring data;
- on deployment boundaries, through network isolation and reverse-proxy policies.

Likewise, no cryptographic protocol is a conceptual prerequisite of the current system model. Confidentiality and strong identity guarantees are therefore outside the present scope and remain future extensions rather than core design commitments.

4 Implementation

Please report here all the implementation (technology-dependent) choices you made while implementing your design.

> If you run out of time for this project, you may consider leaving some aspect unimplemented, and simply discuss how you would have implemented them. In this case, better would be to discuss unimplemented features in section 9.

- which **network protocols** to use?
  - e.g. UDP, TCP, HTTP, WebSockets, gRPC, XMPP, AMQP, MQTT, etc.
- how should *in-transit data* be **represented**?
  - e.g. JSON, XML, YAML, Protocol Buffers, etc.
- how should *databases* be **queried**?
  - e.g. SQL, NoSQL, etc.
- how should components be *authenticated*?
  - e.g. OAuth, JWT, etc.
- how should components be *authorized*?
  - e.g. RBAC, ABAC, etc.

4.1 Technological details

- any particular *framework / technology* being exploited goes here

5 Validation

5.1 Automatic Testing

- how were individual components ***unit*-test**ed?
- how was communication, interaction, and/or integration among components tested?
- how to ***end-to-end*-test** the system?
  - e.g. production vs. test environment
- for each test specify:
  - rationale of individual tests
  - how were the test automated
  - how to run them
  - which requirement they are testing, if any

  recall that *deployment* **automation** is commonly used to *test* the system in *production-like* environment

  recall to test corner cases (crashes, errors, etc.)

5.2 Acceptance test

- did you perform any *manual* testing?
  - what did you test?
  - why wasn't it automatic?

6 Deployment

- should one install your software from scratch, how to do it?
  - provide instructions
  - provide expected outcomes
- what software should be installed on the machines to run your project? which versions?
- should one set environment variables or configuration files?
- if you're using containerization (e.g. Docker), describe how you engineered your deployment scrips (e.g. `docker-compose.yml` files)

7 User Guide

- how to use your software?
  - provide instructions
  - provide expected outcomes
  - provide screenshots if possible

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

Possible future extensions include stronger security mechanisms, persistent storage for historical sensor data, richer external integrations, and broader experimentation with larger topologies or heterogeneous sensor sources.

Another useful direction would be to integrate more advanced monitoring and analysis capabilities, including long-term trend collection, alerting, and comparative evaluation of alternative dissemination or merge strategies.